Mathematical Validation and Critique of THORN

Transport-aligned Heterogeneous Observer-Routed Neighborhood Attention (THORN++)

1 Preliminaries and Notation

Let $G^{(m)} = (V, E^{(m)})$ denote the m-th neighborhood view over a common node set V (e.g., $m \in \{\text{kNN}, \text{time}, \text{diff}\}$). For each view, let $A^{(m)} \in \mathbb{R}_{\geq 0}^{|V|\times|V|}$ be a (possibly weighted) adjacency operator. We write i for the *central* node and j for a *neighbor* of i; attention weights are normalized over j:

$$\alpha_{ij}^{(h)} \propto \exp(\cdot), \qquad \sum_{j\in\mathcal{N}(i)} \alpha_{ij}^{(h)} = 1.$$

In implementation (PyTorch Geometric convention), we store directed edges as $(j \to i)$ in `edge_index`, so `edge_index[0]` is j (source/neighbor) and `edge_index[1]` is i (target/central). This matches the mathematical direction "gather from j into i."

The router produces routing probabilities over views for each node-head pair:

$$\pi_{i,h,m} = \frac{\exp(\ell_{i,h,m})}{\sum_{m'} \exp(\ell_{i,h,m'})}, \qquad \sum_m \pi_{i,h,m} = 1, \quad \pi_{i,h,m} \geq 0.$$

Density-invariant view mixing. Multi-view neighborhoods can differ drastically in degree statistics. To prevent the router from learning a spurious preference for sparse views (because attention mass is less diluted), we mix *degree-normalized* view operators with an explicit clamp for numerical safety:

$$\bar{\deg}_i^{(m)} = \max\big(\deg_i^{(m)}, \epsilon_{\deg}\big), \qquad \deg_i^{(m)} = \sum_j A_{ij}^{(m)}, \qquad \hat{A}_{ij}^{(m)} = \frac{A_{ij}^{(m)}}{\sqrt{\bar{\deg}_i^{(m)}\, \bar{\deg}_j^{(m)}}}.$$

The routed edge mass used by head h at node i is then

$$\tilde{A}_{ij}^{(i,h)} = \sum_m \pi_{i,h,m}\, \hat{A}_{ij}^{(m)}. \tag{1}$$

Remark 1 (Implementation reminder: correct direction, stable normalization, and self-loop policy)**.** *In PyTorch Geometric,* ***`edge_index[0]`*** *denotes the* source *(neighbor) j and* ***`edge_index[1]`*** *denotes the* target *(central) i. Since THORN aggregates $j \to i$, both degree computation (for $\deg_i^{(m)}$) and neighborhood-softmax must use the target index i.*

Self-loops should be enforced before *constructing E_{union} and any aligned edge-feature tensor e_{ij}. Adding self-loops inside a layer is safe only if edge features are extended consistently; otherwise,* ***`edge_index`*** *and* ***`edge_feat`*** *become misaligned.*

```
# Convention: edge_index stores directed edges (j -> i)
j, i = edge_index[0], edge_index[1]

# Degree for normalization: deg_i = sum_{j:(j->i)} A_ij
deg = torch.zeros(N, device=edge_weight.device, dtype=edge_weight.dtype)
deg.scatter_add_(0, i, edge_weight)
deg = deg.clamp_min(eps_deg) # eps_deg ~ 1e-12 (fp32) or 1e-6 (fp16)
inv_sqrt = deg.rsqrt() # stable 1/sqrt(deg)
edge_weight_hat = edge_weight * inv_sqrt[i] * inv_sqrt[j] # \hat A_ij
```

Log-barrier (soft) masking. THORN applies a differentiable relaxation of hard attention masks via

$$\alpha_{ij}^{(h)} \;=\; \mathrm{softmax}_j\Big(s_{ij}^{(h)} + b_{ij}^{(h)} + \log(\tilde{A}_{ij}^{(i,h)} + \epsilon)\Big), \tag{2}$$

where $b_{ij}^{(h)}$ is an optional edge bias computed from edge features and $\epsilon > 0$ is a numerical stabilizer.

Remark 2 (Implementation reminder: attention direction and log-barrier masking)**.** *On a union edge list with PyG orientation $(j \to i)$, compute q_i from the target index and (k_j, v_j) from the source index, normalize over neighbors of each target i, then scatter-add messages to i.*

```
# Q, K, V: [N, H, d]; edge_index_u: [2, E] stores (j -> i)
j, i = edge_index_u[0], edge_index_u[1]

scores = (Q[i] * K[j]).sum(-1) / math.sqrt(d) # [E, H] : s_ij^(h)
scores = scores + edge_bias # b_ij^(h), shape [E, H]

log_mask = (A_tilde + eps).log() # log(\tilde A_ij + eps), [E, H] or [E, 1]
logits = scores + log_mask

attn = softmax_over_neighbors(logits, i) # softmax over j for each fixed i
msg = attn.unsqueeze(-1) * V[j] # [E, H, d]
out = scatter_add(msg, i, dim=0, dim_size=N) # aggregate into node i
```

2 Pillar I: Validity of Log-Barrier Masking

Proposition 1 (Continuous relaxation of sparse masking)**.** *Assume $\tilde{A}_{ij}^{(i,h)} \in [0,1]$ and $\epsilon \downarrow 0$. Then (2) is a continuous relaxation of discrete sparse attention: if $\tilde{A}_{ij}^{(i,h)} = 0$ then $\alpha_{ij}^{(h)} \to 0$, while if $\tilde{A}_{ij}^{(i,h)} > 0$ then $\alpha_{ij}^{(h)}$ is well-defined and differentiable with respect to $\tilde{A}_{ij}^{(i,h)}$.*

Proof. If $\tilde{A}_{ij}^{(i,h)} \to 0$, then $\log(\tilde{A}_{ij}^{(i,h)} + \epsilon) \to -\infty$, so $\exp(s_{ij}^{(h)} + b_{ij}^{(h)} + \log(\tilde{A}_{ij}^{(i,h)} + \epsilon)) \to 0$ and hence the softmax assigns vanishing mass to (i,j). If $\tilde{A}_{ij}^{(i,h)} > 0$, then $\log(\tilde{A}_{ij}^{(i,h)} + \epsilon)$ is finite and smooth; since softmax is smooth, $\alpha_{ij}^{(h)}$ is differentiable in $\tilde{A}_{ij}^{(i,h)}$ and therefore in router logits $\ell_{i,h,m}$ through (1). □

Relation to prior work. Graph Attention Networks enforce sparsity by restricting $j \in \mathcal{N}(i)$ via hard indexing or binary masks; Transformers commonly implement masking by adding $-\infty$ to disallowed logits. The log-barrier formulation retains the semantics of hard masking while remaining differentiable with respect to routing and view mixing.

3 Pillar II: Strict Generalization (Subsumption) of Existing Architectures

Theorem 1 (THORN strictly subsumes standard graph-attention and graph-transformer baselines)**.** *Under suitable parameter settings, THORN reduces to (i) multi-head neighborhood graph attention, (ii) fully-connected transformer attention with structural biases, and (iii) Mixture-of-Experts (MoE) routing as special cases.*

Sketch via reductions. **(i) GAT reduction.** Let $M = 1$ (single view), set $\pi_{i,h,1} = 1$ for all (i, h), and set $b_{ij}^{(h)} = 0$. Then $\tilde{A}_{ij}^{(i,h)} = \hat{A}_{ij}^{(1)}$ and (2) implements masked neighborhood attention with multi-head aggregation.

(ii) Graphormer reduction. Let $A^{(1)}$ be fully connected so $\tilde{A}_{ij}^{(i,h)} \equiv 1$ and the log term is constant. If $b_{ij}^{(h)}$ encodes shortest-path distance or other structural encodings, (2) reduces to transformer attention with graph-structural biases.

(iii) MoE reduction. Replace graph "views" by independent expert modules and interpret $\pi_{i,h,m}$ as the expert gating distribution. Then (1) becomes mixture-of-experts selection. □

4 Pillar III: Differentiability of Transport-Aligned Routing

Let $D_{i,m}$ denote a (precomputed or slowly updated) discordance cost of view m relative to a fused/consensus structure at node i (e.g., ego-network FGW discrepancy). We regularize the router by the expected discordance:

$$\mathcal{L}_{\text{align}} \;=\; \sum_i \sum_h \sum_m \pi_{i,h,m}\, D_{i,m}. \tag{3}$$

Proposition 2 (Gradient form and implicit baseline)**.** *Let $\pi_{i,h,\cdot}$ be a softmax over logits $\ell_{i,h,\cdot}$. Then*

$$\frac{\partial \mathcal{L}_{align}}{\partial \ell_{i,h,m}} \;=\; \pi_{i,h,m}\Big(D_{i,m} - \sum_k \pi_{i,h,k}\, D_{i,k}\Big).$$

Proof. Using $\frac{\partial \pi_k}{\partial \ell_m} = \pi_k(\delta_{km} - \pi_m)$ and the chain rule:

$$\frac{\partial \mathcal{L}_{\text{align}}}{\partial \ell_{i,h,m}} = \sum_k D_{i,k} \frac{\partial \pi_{i,h,k}}{\partial \ell_{i,h,m}} = \sum_k D_{i,k}\, \pi_{i,h,k}(\delta_{km} - \pi_{i,h,m}) = \pi_{i,h,m}\Big(D_{i,m} - \sum_k \pi_{i,h,k} D_{i,k}\Big).$$

□

5 Geometric and Numerical Critique Integrated into THORN++

5.1 Degree–density imbalance and entropy gaming

If attention normalization is performed over the union neighborhood without compensating for view density, the router may be biased toward sparser views because they concentrate attention mass on fewer neighbors, yielding an easier low-entropy optimum unrelated to true manifold geometry. The degree-normalized mixing in (1) mitigates this by rescaling each view operator so that the effective contribution of a view is governed by structural compatibility rather than raw neighborhood size.

When per-view normalization is preferred, a density-invariant alternative computes attention separately on each view neighborhood and mixes post-softmax:

$$\alpha_{ij}^{(h,m)} = \mathrm{softmax}_{j\in\mathcal{N}^{(m)}(i)}\big(s_{ij}^{(h)} + b_{ij}^{(h,m)}\big), \qquad y_i^{(h)} = \sum_m \pi_{i,h,m} \sum_{j\in\mathcal{N}^{(m)}(i)} \alpha_{ij}^{(h,m)} v_j^{(h)}.$$

5.2 Head collapse and geometry diversity

Without explicit incentives, the router can collapse: all heads may select the same view (often kNN) because it yields immediate feature smoothing and stable gradients. THORN++ incorporates diversity and load-balancing in the objective.

Let $P_i \in \mathbb{R}^{H\times M}$ be the matrix whose (h,m) entry is $\pi_{i,h,m}$. A head-view orthogonality penalty discourages redundant routing across heads:

$$\mathcal{L}_{\mathrm{orth}} = \lambda_{\mathrm{orth}} \sum_i \left\| \frac{P_i P_i^\top}{\|P_i\|_F^2} - \frac{I_H}{H} \right\|_F^2.$$

In addition, a global view-usage regularizer prevents batch-level collapse:

$$u_m = \frac{1}{NH}\sum_{i,h} \pi_{i,h,m}, \qquad \mathcal{L}_{\mathrm{lb}} = \lambda_{\mathrm{lb}} \sum_{m=1}^{M} \left(u_m - \frac{1}{M}\right)^2.$$

Remark 3 (Implementation reminder: squared Frobenius norm for $\mathcal{L}_{\mathrm{orth}}$)**.** *To match $\|\cdot\|_F^2$ exactly, ensure the Frobenius norm is* squared *(or compute the elementwise square sum directly). Using an un-squared matrix norm changes the gradient geometry.*

```
# pi: [N, H, M]
# For each i, P = pi[i] has shape [H, M]
P = pi[i] # [H, M]
G = P @ P.T # [H, H]
den = (P * P).sum().clamp_min(eps) # ||P||_F^2
C = G / den - torch.eye(H, device=P.device)/H
L_orth_i = (C * C).sum() # == ||C||_F^2

# Alternatively:
# L_orth_i = torch.linalg.matrix_norm(C, ord='fro').pow(2)
```

5.3 Static vs. dynamic manifold signal (LID semantics)

Local intrinsic dimension (LID) computed once on raw features should be interpreted as a structural prior, not as an invariant of learned representations across layers. THORN++ therefore treats LID_i as an input observer signal unless a deliberate layerwise proxy is introduced and recomputed periodically (with stop-gradient through neighbor selection).

5.4 Transport alignment at scale (ego-alignment and caching)

Global fused Gromov–Wasserstein alignment is computationally prohibitive at scale. THORN++ adopts a local alignment strategy: compute transport discrepancies on ego-neighborhoods and cache them as node/edge consensus features (refreshed slowly offline). During forward passes and at inference, transport enters only as a static (or slowly updated) observer signal $D_{i,m}$.

6 THORN++ Pseudocode

Algorithm 1 summarizes an end-to-end THORN++ pipeline, including (i) offline multi-geometry neighborhood construction and cached alignment, and (ii) routed neighborhood attention with degree-normalized view mixing and collapse-robust regularization.

References (to be completed)

Include citations for: Transformer attention; GAT; Graphormer; MoE/Switch routing; diffusion maps; LID estimation; FGW/GW for graphs.

Algorithm 1 THORN++: Transport-aligned Heterogeneous Observer-Routed Neighborhood Attention

Require: Raw node features $X \in \mathbb{R}^{N \times d_0}$, timestamps $t \in \mathbb{R}^N$ (optional), number of heads H, views $\mathcal{M} = \{\text{kNN}, \text{time}, \text{diff}\}$ with $M = |\mathcal{M}|$, depth L

Require: Hyperparameters: k (kNN), time window/lag parameters, diffusion scale parameters, $\epsilon > 0$ (log-barrier), $\epsilon_{\text{deg}} > 0$ (degree clamp), regularization weights $\lambda_{\text{orth}}, \lambda_{\text{lb}}, \lambda_{\text{gw}}$

Offline preprocessing (static priors and cached alignment)

Build A^{kNN} and distances d_{ij}^{kNN}

Build A^{time} and edge features d_{ij}^{time} (e.g., $|t_i - t_j|$)

Compute diffusion coordinates/affinities, yielding A^{diff} and features d_{ij}^{diff}

Estimate LID_i from the base space and/or A^{kNN}; treat as a structural prior

(Optional) Compute ego-network alignment scores between views and cache as consensus features g_{ij} (no online OT at inference)

Ensure self-loops in every view *before* union construction and edge-feature alignment: for all $m \in \mathcal{M}$ and all i, set $(i, i) \in E^{(m)}$

Form union edge set $E_{\text{union}} = \bigcup_{m \in \mathcal{M}} E^{(m)}$ with per-view weights $A_{ij}^{(m)}$ defined on E_{union}

For each view m, compute degrees $\deg_i^{(m)}$ and clamp $\bar{\deg}_i^{(m)} \leftarrow \max(\deg_i^{(m)}, \epsilon_{\text{deg}})$; define $\hat{A}_{ij}^{(m)} \leftarrow \frac{A_{ij}^{(m)}}{\sqrt{\bar{\deg}_i^{(m)} \bar{\deg}_j^{(m)}}}$

Define edge observer features $e_{ij} \leftarrow [d_{ij}^{\text{kNN}}, d_{ij}^{\text{time}}, d_{ij}^{\text{diff}}, \text{LID}_i, \text{LID}_j, g_{ij}]$

Training / inference (forward pass)

Initialize hidden states $H^{(0)} \leftarrow \text{Embed}(X)$

for $\ell = 1$ to L **do**

  Compute query/key/value projections per head: $Q^{(\ell)}, K^{(\ell)}, V^{(\ell)}$ from $H^{(\ell-1)}$

  Compute router context with an MPNN over E_{union}: $C^{(\ell)} \leftarrow \text{MPNN}(H^{(\ell-1)}, E_{\text{union}}, e)$

  For each node i and head h, produce routing logits $\ell_{i,h,m}^{(\ell)}$ over views $m \in \mathcal{M}$

  Routing probabilities: $\pi_{i,h,m}^{(\ell)} \leftarrow \text{softmax}_m(\ell_{i,h,m}^{(\ell)})$

  Edge bias per head: $b_{ij}^{(\ell,h)} \leftarrow \phi_h^{(\ell)}(e_{ij})$

  Degree-normalized routed mask on edges:

$$\tilde{A}_{ij}^{(\ell,i,h)} \leftarrow \sum_{m \in \mathcal{M}} \pi_{i,h,m}^{(\ell)} \hat{A}_{ij}^{(m)}$$

  Attention logits on E_{union} (edge list stores $(j \to i)$):

$$z_{ij}^{(\ell,h)} \leftarrow \frac{\langle q_i^{(\ell,h)}, k_j^{(\ell,h)} \rangle}{\sqrt{d}} + b_{ij}^{(\ell,h)} + \log(\tilde{A}_{ij}^{(\ell,i,h)} + \epsilon)$$

  Neighborhood attention weights: $\alpha_{ij}^{(\ell,h)} \leftarrow \text{softmax}_{j:(j \to i) \in E_{\text{union}}}(z_{ij}^{(\ell,h)})$

  Aggregate per head: $y_i^{(\ell,h)} \leftarrow \sum_{j:(j \to i) \in E_{\text{union}}} \alpha_{ij}^{(\ell,h)} v_j^{(\ell,h)}$

  Combine heads and apply residual/FFN: $H^{(\ell)} \leftarrow \text{Block}(H^{(\ell-1)}, \text{Concat}_h\, y^{(\ell,h)})$

end for

Compute task loss $\mathcal{L}_{\text{task}}$ from $H^{(L)}$

Head-view orthogonality: $\mathcal{L}_{\text{orth}} \leftarrow \sum_i \left\| \frac{P_i P_i^\top}{\|P_i\|_F^2} - \frac{I_H}{H} \right\|_F^2$

View load-balancing: $\mathcal{L}_{\text{lb}} \leftarrow \sum_{m=1}^{M} \left(u_m - \frac{1}{M}\right)^2$

  where $u_m = \frac{1}{NH} \sum_{i,h} \pi_{i,h,m}$

(Optional) Alignment penalty: $\mathcal{L}_{\text{gw}} \leftarrow \sum_{i,h,m} \pi_{i,h,m} D_{i,m}$

Total loss: $\mathcal{L} \leftarrow \mathcal{L}_{\text{task}} + \lambda_{\text{orth}} \mathcal{L}_{\text{orth}} + \lambda_{\text{lb}} \mathcal{L}_{\text{lb}} + \lambda_{\text{gw}} \mathcal{L}_{\text{gw}}$

Update parameters by backpropagation